

Versionamento e Evolução de APIs REST

APIs bem-sucedidas raramente permanecem estáticas. Novas funcionalidades, novos consumidores e novas necessidades exigem evolução constante. Aprenda a versionar e evoluir suas APIs REST com segurança e estratégia.

Objetivos de Aprendizagem

Ao final deste material, você será capaz de compreender e aplicar as principais práticas de versionamento de APIs REST no mundo real.

Identificar a necessidade

Compreender quando uma API precisa ser versionada e por quê.

Classificar mudanças

Identificar mudanças compatíveis e incompatíveis com precisão.

Implementar no NestJS

Configurar e criar versões de controllers no NestJS.

Planejar a evolução

Evoluir a API sem quebrar aplicações existentes.

Múltiplas versões

Trabalhar com v1, v2 e outras versões simultaneamente.

Depreciar endpoints

Aplicar estratégias seguras de depreciação e remoção.

O Problema da Evolução

No início de um projeto, uma API normalmente possui poucos consumidores. Com o tempo, novos sistemas passam a depender dela — e qualquer alteração precisa ser planejada cuidadosamente.

Início do projeto

A API serve apenas um consumidor:

- Frontend Web

Alterações são simples e de baixo risco.

Com o crescimento

A API passa a ser consumida por múltiplos sistemas:

- Frontend Web

- Aplicativo Mobile
- Dashboard Administrativo
- Integrações Externas

Qualquer mudança pode impactar todos esses consumidores simultaneamente.

Quanto mais consumidores uma API possui, maior o risco de uma alteração não planejada causar interrupções em produção.

O que Pode Quebrar uma Integração?

Mudanças aparentemente simples podem gerar impactos significativos. Considere este exemplo clássico de renomeação de campo:

Resposta original (v1)

```
{
  "id": 1,
  "name": "Maria"
}
```

O frontend lê o campo `name` sem problemas.

Após a alteração

```
{
  "id": 1,
  "fullName": "Maria"
}
```

O backend continua funcionando. O frontend pode parar **imediatamente**.

Renomear um campo é uma das mudanças mais perigosas em uma API pública. O backend não apresenta erro, mas todos os consumidores que dependem do campo antigo são silenciosamente quebrados.

Mudanças Compatíveis vs. Incompatíveis

Nem toda mudança exige uma nova versão. Saber classificar corretamente é essencial para evitar versionamento desnecessário.

✓ Mudanças Compatíveis

Não exigem nova versão. Clientes antigos continuam funcionando.

- Adicionar novo campo na resposta
- Adicionar novo endpoint
- Adicionar filtro opcional
- Adicionar paginação opcional

✓ Mudanças Compatíveis - **continuação**

- Adicionar novos recursos

// Antes

```
{ "id": 1, "name": "Maria" }
```

// Depois (compatível)

```
{ "id": 1, "name": "Maria", "phone": "(45)99999-9999" }
```

✗ Mudanças Incompatíveis

Geralmente exigem nova versão. Clientes antigos deixarão de funcionar.

- Renomear campo existente
- Remover campo existente
- Alterar tipo de dado de um campo
- Mudar estrutura da resposta
- Alterar comportamento de endpoint

// Antes

```
{ "name": "Maria" }
```

// Depois (incompatível)

```
{ "firstName": "Maria" }
```

Quando Criar uma Nova Versão?

Versionamento deve ser uma **decisão estratégica**, não uma reação automática a qualquer mudança. Não é necessário criar v2, v3, v4 para cada pequena alteração.

Crie uma nova versão quando...

- Mudanças quebram consumidores atuais
- A manutenção da versão atual se tornou inviável
- Há uma reestruturação significativa da API

Não crie uma nova versão para...

- Adição de campos opcionais
- Novos endpoints independentes
- Correções de bugs sem mudança de contrato
- Melhorias de performance internas

Versionar em excesso aumenta a complexidade de manutenção sem trazer benefícios reais. Prefira evoluir a API de forma compatível sempre que possível.

Estratégias de Versionamento

Existem três abordagens principais para versionar uma API REST. Cada uma tem suas vantagens e casos de uso ideais.

1

Versionamento por URL

A estratégia mais utilizada e recomendada.

```
/api/v1/users  
/api/v2/users
```

- Fácil entendimento
- Fácil manutenção
- Fácil documentação
- Testável diretamente no navegador

2

Versionamento por Header

A versão é informada no cabeçalho da requisição.

```
GET /users  
Api-Version: 2
```

- URL permanece limpa
- Menos intuitivo para desenvolvedores
- Mais difícil de testar manualmente

3

Versionamento por Query String

A versão é passada como parâmetro na URL.

```
/users?version=2
```

- Simples de implementar
- Pouco utilizado em APIs corporativas modernas
- Pode conflitar com outros parâmetros

Habilitando Versionamento no NestJS

O NestJS oferece suporte nativo ao versionamento de APIs. A configuração é feita no arquivo `main.ts` com poucas linhas de código.

Passo 1 — Importar o tipo

No arquivo `main.ts`, importe o `VersioningType`:

```
import { VersioningType } from '@nestjs/common';
```

Passo 2 — Habilitar o versionamento

Configure o versionamento por URI na aplicação:

```
app.enableVersioning({  
  type: VersioningType.URI,  
});
```

Agora o NestJS reconhecerá automaticamente as rotas `/v1` e `/v2`.

Passo 3 — Criar o Controller v1

Defina o controller com a versão desejada:

```
@Controller({  
  path: 'users',  
  version: '1',  
})  
export class UsersV1Controller {}
```

Endpoint gerado: `/v1/users`

Passo 4 — Criar o Controller v2

Crie um segundo controller para a nova versão:

```
@Controller({  
  path: 'users',  
  version: '2',  
})  
export class UsersV2Controller {}
```

Endpoint gerado: `/v2/users`

As duas versões podem coexistir simultaneamente no mesmo projeto NestJS, sem conflitos de rota.

Mantendo Múltiplas Versões e Estratégia de Depreciação

Manter múltiplas versões ativas reduz riscos durante migrações. Uma versão não deve ser removida imediatamente — o processo de depreciação deve ser gradual e comunicado com antecedência.

Coexistência de versões

v1 → Sistema legado
v2 → Novo frontend

Ambos continuam funcionando em paralelo. Isso permite que cada consumidor migre no seu próprio ritmo, sem pressão ou risco de interrupção.

Nunca remova uma versão sem antes confirmar que todos os consumidores migraram.

Fluxo de depreciação recomendado

01	02	03
Criar v2	Comunicar consumidores	Migrar aplicações
Implementar a nova versão com as mudanças necessárias.	Notificar todas as equipes sobre a nova versão e o prazo de migração.	Apoiar as equipes na atualização para a nova versão.
04	05	
Monitorar utilização	Remover v1	
Acompanhar métricas de uso da v1 até que o tráfego cesse.	Somente após confirmação de que nenhum consumidor ativo depende dela.	

Documentação, Boas Práticas e Checklist Final

Uma API bem versionada precisa de documentação clara. O Swagger deve deixar explícito quais versões estão ativas, quais estão obsoletas e quais serão removidas em breve.

Documentando versões no Swagger

Cada versão deve ser documentada separadamente:

/v1/users → obsoleto
/v2/users → recomendado

- Indicar qual versão é recomendada
- Marcar versões obsoletas claramente
- Informar quais serão removidas futuramente

Evitando duplicação excessiva

Nem toda a API precisa ser recriada a cada versão. Apenas o módulo que sofreu mudanças incompatíveis deve receber nova versão:

Users → mudou → v1 e v2
Products → não mudou → apenas v1
Categories → não mudou → apenas v1

Planejando a evolução gradual

v1 → MVP
v2 → autenticação avançada
v3 → novos recursos

O objetivo é permitir crescimento sem interromper consumidores existentes.

Checklist de Validação

- ✓ Conceito de mudança compatível compreendido
- ✓ Conceito de mudança incompatível compreendido
- ✓ Versionamento URI configurado no NestJS
- ✓ Controller versionado criado
- ✓ Múltiplas versões funcionando simultaneamente
- ✓ Estratégia de migração definida
- ✓ Estratégia de depreciação definida
- ✓ Swagger atualizado com versões documentadas

APIs bem-sucedidas raramente permanecem estáticas. O versionamento permite que a evolução ocorra de forma segura, preservando compatibilidade com sistemas existentes e reduzindo riscos de interrupção. Ao planejar corretamente a criação, manutenção e remoção de versões, garantimos que a API continue crescendo sem comprometer aplicações que dependem dela.